LLD - Email Attachment Processing

Low Level Design: Email Attachment Processing

Parent Epic: Email Integration

Story Points: 3 | **Status:** Done | **Sprint:** Backlog

SUMMARY

Problem Statement

Extract and process PDF/Excel attachments from monitored emails, routing them through the OCR/parser pipeline.

User Story

As a payroll auditor, I want email attachments automatically processed so that timecards are extracted without manual intervention.

Business Impact

- Improved efficiency in timecard processing workflow
- Reduced manual data entry and human error
- Better compliance with client-specific audit rules
- Faster payroll processing turnaround

REQUIREMENTS

Requirements

Extract and process PDF/Excel attachments from monitored emails, routing them through the OCR/parser pipeline.

Use Cases and Success Criteria

- PDF attachments extracted and processed
- Excel/CSV attachments extracted and processed
- Attachments routed to appropriate parser
- Confirmation reply sent to sender
- Processed emails marked as read
- Failed processing triggers alert

User Journeys and Flows

```
```

```
User Journey: Email Attachment Processing
+-------------------------------------------------------------+
| 1. User initiates action                                    |
|    v                                                        |
| 2. System processes request                                 |
|    v                                                        |
| 3. Validation and business rules applied                    |
|    v                                                        |
| 4. Data persisted / updated                                 |
|    v                                                        |
| 5. User receives feedback / confirmation                    |
+-------------------------------------------------------------+

```

In Scope

- **Extract PDF/Excel attachments**: Download attachments from email [Done]
- **Route to OCR/parser pipeline**: Send to appropriate processor [Done]
- **Send confirmation reply**: Reply to sender via Graph API [Done]
- **Mark processed emails as read**: Update email status [Done]

Out of Scope

- Features not explicitly listed in acceptance criteria
- Performance optimization beyond baseline requirements
- UI/UX changes outside this story's scope

Assumptions & Constraints

Category	Assumption/Constraint
Technical	FastAPI backend with async SQLAlchemy 2.0
Technical	React 19 frontend with TanStack Query
Business	Client rules are pre-configured before processing
Data	PostgreSQL database with proper indexing
Legal	All data handling complies with data privacy requirements

---

DESIGN

High Level Architecture

```


Component Architecture
======================

+---------------------------------------------------------------------------+
|                                FRONTEND                                   |
```

```
|  +-------------+  +-------------+  +-------------+  +-------------+    |
|  |   Pages     |  | Components  |  |   Hooks     |  |  Services   |    |
|  |  (React)    |  |  (Shadcn)   |  | (TanStack)  |  |   (API)     |    |
|  +-------------+  +-------------+  +-------------+  +-------------+    |
+-------------------------------------------------------------------------+
                                    |
                                    | REST API (HTTPS)
                                    v
+-------------------------------------------------------------------------+
|                              BACKEND                                     |
|  +-------------+  +-------------+  +-------------+  +-------------+    |
|  |   Routers   |  |  Services   |  |   Models    |  |   Engine    |    |
|  |  (FastAPI)  |  |  (Business) |  | (SQLAlchemy)|  |   (Rules)   |    |
|  +-------------+  +-------------+  +-------------+  +-------------+    |
+-------------------------------------------------------------------------+
         |                                     |
         | Redis Queue                         | Async DB
         v                                     v
+-----------------+                   +-----------------+
|     WORKER      |                   |   POSTGRESQL    |
|  +-----------+  |                   |  +-----------+  |
|  | LangGraph |  |                   |  |  Tables   |  |
|  |   Jobs    |  |                   |  |  Indexes  |  |
|  +-----------+  |                   |  +-----------+  |
+-----------------+                   +-----------------+
         |
         | External APIs
         v
+-------------------------------------------------------------------------+
|                        EXTERNAL SERVICES                                 |
|  +-------------+  +-------------+  +-------------+  +-------------+    |
|  |   Avionte   |  |   Outlook   |  |   Gemini    |  |  Entra ID   |    |
|  |  BOLD API   |  | Graph API   |  |  Vision     |  |   (OIDC)    |    |
|  +-------------+  +-------------+  +-------------+  +-------------+    |
+-------------------------------------------------------------------------+


```

Low Level Design / Detailed Design

Sequence Diagram

```


Sequence: Document Processing Flow
==================================

  User          Frontend        Backend         Worker          Gemini
    |               |               |               |               |
    |--Upload PDF-->|               |               |               |
    |               |--POST /upload->|               |               |
    |               |               |--Queue Job--->|               |
    |               |<--202 Accepted-|               |               |
    |<--Processing--|               |               |               |
    |               |               |               |--OCR Request-->|
    |               |               |               |<--OCR Result---|
    |               |               |<--Job Done----|               |
    |               |<--WebSocket----|               |               |
    |<--Complete----|               |               |               |
    |               |               |               |               |


```

Technical Assumptions

- Async/await patterns used throughout backend
- Database transactions managed via SQLAlchemy session
- API follows RESTful conventions

- Authentication via JWT tokens (to be replaced with Entra ID OIDC)

Approach Analysis

Approach	Pros	Cons	Decision
Synchronous Processing	Simple implementation	Blocks user, poor UX for large files	Not Selected
Async with Background Worker	Non-blocking, scalable	More complex, needs Redis	Selected
Event-driven (Kafka)	Highly scalable	Over-engineered for current needs	Not Selected

Open Questions

- TBD: Performance benchmarks for production load
- TBD: Error retry policies fine-tuning

APIs / Contracts

Internal APIs

Method	Endpoint	Description	Auth
GET	/api/v1/entries	List entries with filtering	JWT Required
POST	/api/v1/entries	Create new entry	JWT Required
PUT	/api/v1/entries/{id}	Update entry	JWT Required
POST	/api/v1/entries/{id}/approve	Approve entry	JWT Required

Request/Response Schema

```

// Request Example
{
  "employee_name": "John Doe",
  "client_name": "ABC Corp",
  "week_ending": "2024-01-07",
  "hours": {
    "monday": 8.0,
    "tuesday": 8.0,
    "wednesday": 8.0,
    "thursday": 8.0,
    "friday": 8.0
  }
}

// Response Example
```

```
{
  "id": 123,
  "status": "pending",
  "created_at": "2024-01-08T10:30:00Z",
  "message": "Entry created successfully"
}

```

Authentication / Authorization

- Current: JWT Bearer token in Authorization header
- Future: Microsoft Entra ID OIDC
- Roles: admin, auditor, viewer

Data Models

Database Tables

```

-- Core tables involved in this feature
CREATE TABLE employee_entries (
    id SERIAL PRIMARY KEY,
    batch_id INTEGER REFERENCES batches(id),
    employee_name VARCHAR(255),
    client_name VARCHAR(255),
    week_ending DATE,
    total_hours DECIMAL(5,2),
    status VARCHAR(50) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_entries_status ON employee_entries(status);
CREATE INDEX idx_entries_batch ON employee_entries(batch_id);

```

ER Diagram

```

+-----------------+      +-----------------+      +-----------------+
|     batches     |      | employee_entries|      | client_rules    |
+-----------------+      +-----------------+      +-----------------+
| id (PK)         |--+   | id (PK)         |   +--| id (PK)         |
| filename        |  |   | batch_id (FK)   |---+  | client_name     |
| status          |  +-->| employee_name   |      | ot_threshold    |
| created_at      |      | client_name     |------| ot_multiplier   |
+-----------------+      | week_ending     |      | aca_enabled     |
                         | total_hours     |      +-----------------+
                         | status          |
                         +-----------------+

```

Failure Handling

Failure Scenario	Detection	Recovery Action
API timeout	Request timeout > 30s	Retry with exponential backoff (max 3)

Database connection lost	Connection pool exhausted	Circuit breaker, failover to replica
Invalid input data	Pydantic validation	Return 400 with detailed errors
External API failure	Non-2xx response	Queue for retry, alert on 3 failures

---

Testing Approach

Test Type	Coverage	Tools
Unit Tests	All service functions	pytest, pytest-asyncio
Integration Tests	API endpoints	pytest, httpx
E2E Tests	Critical user flows	Playwright
Performance Tests	Load and stress	k6, Locust

Performance Metrics

Latency Characteristics

Operation	Target P50	Target P99
API Response	< 200ms	< 1s
File Upload	< 2s	< 5s
OCR Processing	< 10s	< 30s
Database Query	< 50ms	< 200ms

Scaling Plan

- Horizontal scaling via Kubernetes pods (target: 3 replicas)
- Database read replicas for query load
- Redis cluster for worker queue scaling

NON-FUNCTIONAL REQUIREMENTS

Category	Requirement	Implementation

Reliability	99.9% uptime	Health checks, auto-restart, circuit breakers
Security	Data encryption	TLS 1.3 in transit, AES-256 at rest
Observability	Full stack monitoring	CloudWatch logs, X-Ray traces, custom metrics
Compliance	Data privacy	PII handling per privacy policy
Scalability	100 concurrent users	Horizontal pod autoscaling
Maintainability	Modular design	Service layer separation, dependency injection

---

FAQs

Question	Answer
How are errors reported to users?	Toast notifications with actionable messages
What happens if OCR fails?	Entry marked for manual review with original document
Can users undo approvals?	Yes, within 24 hours if not synced to Avionte

---

ONBOARDING

Prerequisites to External Systems

- Avionte BOLD API credentials configured
- Microsoft Graph API (Outlook) credentials
- Google Cloud credentials for Gemini Vision
- PostgreSQL database provisioned
- Redis instance for worker queue

Configuration Steps

1. Clone repository and install dependencies
2. Copy .env.example to .env and configure
3. Run database migrations: `alembic upgrade head`

4. Start backend: `uvicorn app.main:app --reload`
5. Start worker: `python -m app.worker.main`
6. Start frontend: `npm run dev`

LAUNCH PLAN

Timelines

Phase	Duration	Deliverable
Development	3 story points	Feature complete
Code Review	1 day	PR approved
QA Testing	2 days	Test cases passed
UAT	2 days	User acceptance
Deployment	1 day	Production release

Post-Production Monitoring

- Monitor error rates for 48 hours post-deployment
- Track latency metrics against baseline
- Review user feedback and support tickets
- Rollback plan ready if issues detected

REFERENCES

- SOW Gap Analysis: /docs/SOW_GAP_ANALYSIS.md
- Sprint Plan: /docs/SPRINT_PLAN.md
- AWS Deployment Guide: /docs/AWS_Deployment_Guide_Complete.html
- Avionte BOLD API Documentation
- Microsoft Graph API Documentation

Document generated for Clockwork Audit Pro (Verstela) | Last Updated: 2026-02-02